

Static Members vs Instance Members

First, the mental model:

```
Class
|
|— Instance members → belong to EACH OBJECT
|
|— Static members   → belong to the CLASS
```

1. Instance member

Normal data members are **instance members**.

```
class Student {
public:
    int age;
};
```

If we create:

```
Student s1;
Student s2;
```

there are **two separate** **age** variables:

```
s1 → age = ?
s2 → age = ?
```

So:

```
s1.age = 20;
s2.age = 25;
```

doesn't affect the other object.

Storage

The **age** is stored **inside each object**.

```
Memory

s1
┌   │   │   │
│ age = 20 │
└   │   │   │

s2
┌   │   │   │
│ age = 25 │
└   │   │   │
```

Therefore:

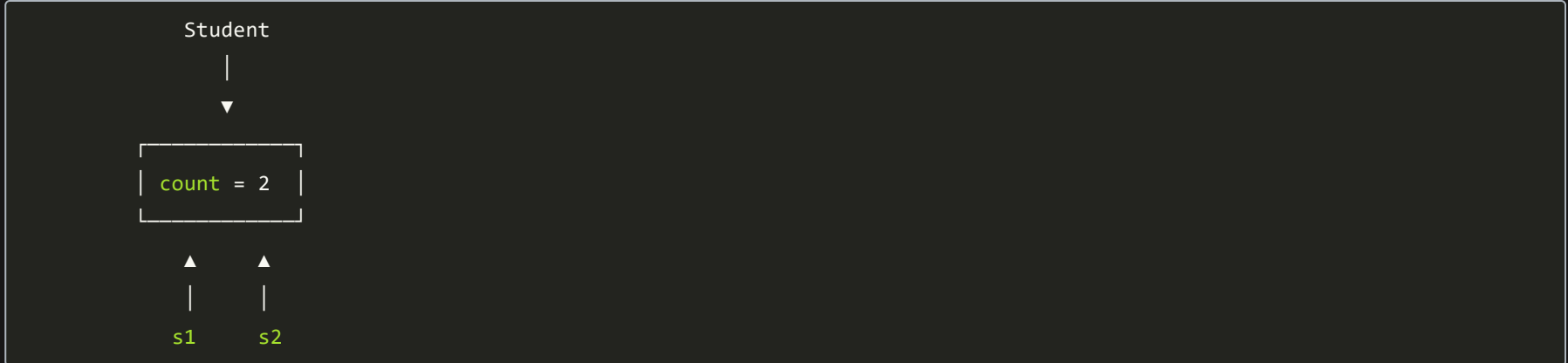
Every object gets its own copy of non-static data members.

2. Static data member

Now:

```
class Student {
public:
    static int count;
};
```

There is **only one** **count**, shared by all objects.



So:

```
Student s1;
Student s2;

Student::count = 10;
```

Both objects conceptually refer to the same **count**.

You normally access it using:

```
Student::count
```

rather than through an object.

3. Why use static members?

A classic example is counting objects:

```
class Student {
public:
    static int count;

    Student() {
        count++;
    }
};

int Student::count = 0;
```

Now:

```
Student s1;
Student s2;
Student s3;

cout << Student::count;
```

Output:

```
3
```

Because there is only **one shared counter**.

Interview mental model

```
Instance member:
    one copy PER OBJECT

Static member:
    one shared copy PER CLASS
```

4. Important: static data member storage

This is a common interview point.

Consider:

```
class Student {
    int age;
    static int count;
};
```

Conceptually:

Student **object 1**

```
age
```

Student **object 2**

```
age
```

Class-level storage

```
count
```

count is not stored separately inside every object.

That's why:

```
sizeof(Student)
```

doesn't increase because of the static data member in the way a normal **int** member would.

5. Static member needs a definition

Historically, if you declare:

```
class Student {
public:
    static int count;
};
```

you also need a definition:

```
int Student::count = 0;
```

Otherwise, if you odr-use it, you'll generally get a linker error.

Modern C++ alternative

With C++17, you can use **inline static**:

```
class Student {
public:
    inline static int count = 0;
};
```

Now you don't need a separate definition.

For interviews, knowing both is useful.

6. Now the important question: static function

Consider:

```
class Student {
    int age;

public:
    static void display() {
        cout << age; // ✗
    }
};
```

Why is this illegal?

Because a **static member function** does not have a **this** pointer.

Remember from the previous topic:

Non-static **function**
↓
has **this**
↓
knows which **object called** it

Static **function**
↓
NO **this**
↓
no particular **object**

So if you write:

```
s1.display();
```

a static function doesn't have an object-specific **this** to determine which **age** you mean.

7. Can static functions access non-static members?

Directly: **✗** No

```
class Student {  
    int age;  
  
public:  
    static void show() {  
        cout << age;    // ✗  
    }  
};
```

The compiler doesn't know which object's **age** you want.

But can it access one through an object?

Yes.

This is an important distinction.

```
class Student {  
    int age;  
  
public:  
    static void show(Student& s) {  
        cout << s.age;    // ✓  
    }  
};
```

Now:

```
Student s;  
Student::show(s);
```

works because you explicitly provided the object.

The static function itself still doesn't have **this**.

8. Static functions can access static members

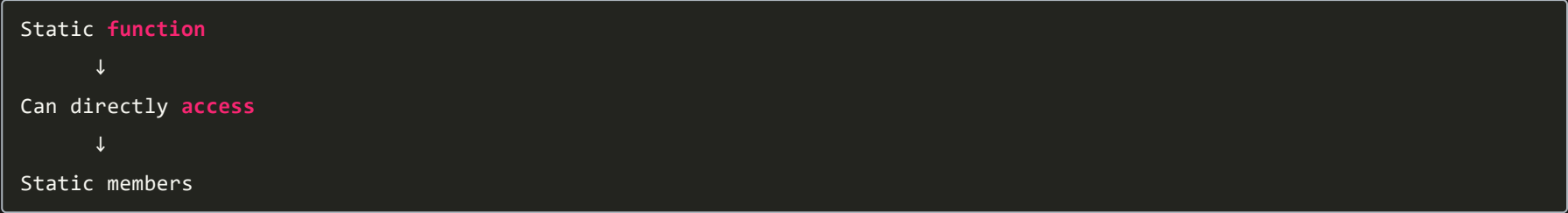
Of course:

```
class Student {  
    static int count;  
  
public:  
    static void showCount() {  
        cout << count;    // ✓  
    }  
};
```

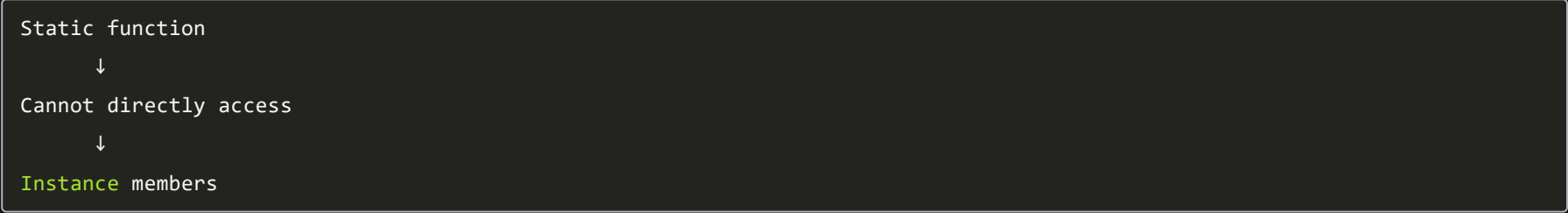
Why?

Because `count` belongs to the **class**, not a particular object.

So:



But:



9. Instance functions can access both

A normal member function has `this`.

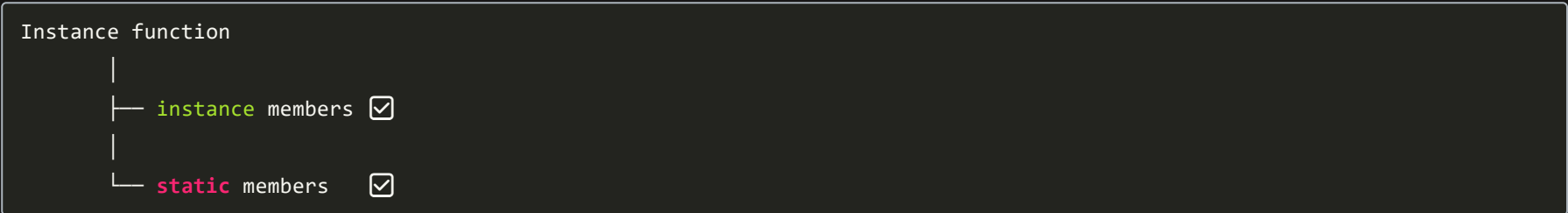
Therefore:

```
class Student {
    int age;
    static int count;

public:
    void show() {
        cout << age;    // ☒ instance member
        cout << count; // ☒ static member
    }
};
```

An instance function can access **both**.

Think:



10. Master table ★

Instance member	Static member	Belongs to	Object	Class	Copies	One per object	One shared copy	Stored inside object?	Yes	No	Requires object?
Yes	No	Access using <code>obj.xClass::x</code>	Accessible by instance function?	☒	☒	Accessible directly by static function?	☒	Has <code>this</code> ?	☒	Instance function:	☒
			Static function:	☒							☒

11. A very common interview question

What happens here?

```
class Test {
public:
    int x = 10;
    static int y;

    static void foo() {
        cout << x;
        cout << y;
    }
};
```

Answer:

```
cout << x; // ☒
cout << y; // ☐
```

Because:

```
x → belongs to an object
y → belongs to the class
```

And `foo()` has no `this`

12. Why doesn't static function have `this`?

Suppose:

```
class Student {  
public:  
    static void hello();  
};
```

You can call:

```
Student::hello();
```

No object exists at all.

So what would `this` point to?

Nothing.

That's the fundamental reason:

A static member function belongs to the class rather than an individual object, so it has no implicit `this` pointer.

13. Interview trap: Can I call a static function using an object?

Yes.

```
Student s;  
  
s.hello();    // ☒
```

But this doesn't make `hello()` an instance function.

It is still static.

Better style:

```
Student::hello();
```

because it makes the intent clear.

14. Classic example: object counter

This is worth knowing for interviews:

```
class Student {  
private:  
    inline static int count = 0;  
  
public:  
    Student() {  
        count++;  
    }  
  
    static int getCount() {  
        return count;  
    }  
};
```

Then:

```
Student s1;  
Student s2;  
Student s3;  
  
cout << Student::getCount();
```

Output:

```
3
```

Here:

count
↓
one **shared** variable



★ Interview cheat sheet

Memorize these:

INSTANCE MEMBER

↓

belongs **to object**
one **copy** per **object**
accessed **using object**
instance **function** can **access** it

STATIC MEMBER

↓

belongs **to class**
one shared **copy**
accessed **using Class::**
no this pointer

STATIC **FUNCTION**

↓

belongs **to class**
no this pointer
can directly **access** static members
cannot directly **access** non-static members
can **access** non-static members **if** given an **object**/reference/pointer

Most likely interview questions

1. What is the difference between static and non-static members?
2. How many copies of a static data member exist?
3. Where is a static data member stored?
4. Can a static function access non-static members? Why not?
5. Does a static function have a **this** pointer?
6. Can a static function access static members?
7. Can a static function be called using an object?
8. Why would you use a static member?

The key connection to remember is:

this represents a specific object. Static members/functions don't belong to a specific object, so static functions don't have **this** and cannot directly access instance members.